

Customer Churn Prediction on the IBM Telco Dataset

Romina Gharehmohammadi

Department of Computer Science and Engineering, DISI

University of Bologna, Cesena Campus

Course: Machine Learning, Prof. Matteo Francia

Email: romi.gharehmohammadi@studio.unibo.it

Abstract—Customer churn is one of the main sources of lost revenue for subscription telecom companies. This work builds a churn prediction pipeline on the public IBM Telco Customer Churn dataset (7,043 customers, 21 features, about 27% churn). Following the CRISP-DM methodology, five classifiers are compared: a Dummy baseline, Logistic Regression, a Decision Tree, a Random Forest, and a Multi-Layer Perceptron. The class imbalance is handled with `class_weight=balanced` in the models that support it, and hyperparameters are selected with 5-fold cross-validation on the training set using F1 on the positive class. On a held-out test set, Logistic Regression and Random Forest reach the best F1 (around 0.62) with a ROC-AUC of about 0.84, which is in line with the results recently reported on the same dataset. The strongest predictors of churn are the contract type, the customer tenure, the internet service and the payment method.

1. Introduction

The telecommunications industry sells most of its services through subscription contracts, so its revenue depends directly on customer retention. When a customer cancels the subscription (churn), the company loses both the future income from that customer and the cost paid to acquire them in the first place. Industry reports usually estimate that acquiring a new customer costs five to seven times more than keeping an existing one, which makes churn prediction a high-value business problem. If the company can identify in advance which customers are likely to leave, the retention team can focus targeted offers on them, instead of sending broad and expensive campaigns to the whole customer base.

In real settings, the retention team typically operates with a fixed budget for offers and contacts, so it cannot reach out to every customer. A predictive model is therefore most useful when it can rank customers by churn risk: the budget can then be spent on the most exposed customers first. For this reason the project pays attention not only to the classification accuracy of the models, but also to their ability to produce well-calibrated risk scores and to identify the features that are actually driving the predictions.

From a machine learning point of view, churn prediction is a binary classification problem. Given a customer profile (demographics, account information, subscribed services),

the goal is to predict whether the customer will cancel within a reference period. Two characteristics of the data make the task non-trivial. First, in practice the churn rate is usually well below 50%, so the classes are imbalanced and accuracy alone is a misleading evaluation metric. Second, most of the available features are categorical (contract type, payment method, optional services), which requires careful encoding before the data can be fed to most classifiers.

This paper builds a complete churn prediction pipeline on the IBM Telco Customer Churn dataset [1], a public benchmark of 7,043 customers and 21 features that has been widely used in the recent literature. The work follows the CRISP-DM methodology [2] and compares five classifiers: a Dummy baseline (majority class), Logistic Regression, a Decision Tree, a Random Forest, and a small Multi-Layer Perceptron. Hyperparameters are chosen with grid search and 5-fold cross-validation on the training set. The class imbalance is handled with the `class_weight=balanced` option in the classifiers that support it.

The main contributions of this work are the following. First, a reproducible end-to-end pipeline on the IBM Telco dataset, executable on Google Colab without errors. Second, a comparison of five classifiers that cover different model families: linear (Logistic Regression), tree (Decision Tree), ensemble (Random Forest), and neural (MLP), together with a Dummy baseline. Third, a short analysis of which customer attributes drive churn, with practical suggestions for the retention team.

As required by the assignment specification, the organization of the group is described here: the project is carried out as an *individual assignment*, so the group consists of one person. All the work (data analysis, code development, hyperparameter tuning, writing of the paper and preparation of the presentation) was done by the author alone, with no division of tasks among multiple members.

The rest of the paper is organized as follows. Section 2 reviews related work on churn prediction. Section 3 describes the proposed method, including the data preparation steps and the modeling protocol. Section 4 reports the results on the test set and compares them with the recent literature. Section 5 closes the paper with a short summary, limitations and possible improvements.

2. Related Work

Customer churn prediction has been an active research area for many years, especially in industries based on subscription contracts such as telecom, banking and online services. The general motivation is the same in all these settings: customer acquisition is much more expensive than customer retention, so identifying in advance the customers who are likely to leave gives the company an opportunity to act before they actually do. The problem has been studied with a wide range of techniques, from classical statistical models such as Logistic Regression to modern ensembles and deep learning approaches.

Ahmad et al. [3] provide a recent overview of churn prediction in the telecom industry. They focus on large-scale implementations using big data platforms and they compare several classifiers (Decision Tree, Random Forest, Gradient Boosted Trees and XGBoost) on a real-world telecom dataset of approximately one million customers. They report that the best models reach over 90% test accuracy on their data. The dataset they use is larger and richer in behavioral information than the one used in this paper: it contains call records, usage patterns and customer-care interactions, which explains the higher accuracy they observe. This difference suggests that the performance of churn models depends not only on the algorithm but also, and often more strongly, on the type of features available. On a dataset with only static customer attributes, like the IBM Telco dataset, the practical performance ceiling tends to be lower regardless of the algorithm used.

The IBM Telco Customer Churn dataset used in this paper has become a common benchmark for the task because it is publicly available, well documented and reasonably small. The closest reference to the present work is Lalwani et al. [4], who performed a systematic comparison of classifiers on the same dataset. They preprocessed the data with a feature selection step and tested several classifiers, including Logistic Regression, Naive Bayes, SVM, Decision Tree, Random Forest, AdaBoost and XGBoost. They reported that boosting methods (AdaBoost and XGBoost) achieved the highest test accuracy (around 81–82%) and a ROC-AUC of approximately 0.84, with the other classical classifiers obtaining slightly lower but comparable results. This collection of numbers is used as the main reference point in Section 4 to evaluate the performance of the pipeline presented in this paper.

A common finding in churn-prediction comparative studies is that no single algorithm clearly dominates the others on tabular customer data. Ensemble methods (Random Forest, boosting) usually obtain the best accuracy, but classical models such as Logistic Regression remain competitive when they are properly regularized and the data is appropriately preprocessed. This is consistent with what is observed in the experiments of this paper, where Logistic Regression and Random Forest produce essentially the same F1 and ROC-AUC. The general lesson is that, on this kind of data, the algorithm choice matters less than the data preparation,

the metric definition and the way the class imbalance is handled.

A common issue in churn prediction is the class imbalance. The share of churners is usually well below 50% (about 27% in the dataset used here), so plain accuracy is a misleading metric: a trivial model that always predicts the majority class already reaches an accuracy equal to the class proportion. Burez and Van den Poel [5] discuss several strategies to deal with class imbalance for churn problems. They compare under-sampling, over-sampling and cost-sensitive learning on several real datasets, and they recommend the use of ROC-based metrics instead of accuracy when the classes are imbalanced. In this paper the imbalance is handled with the `class_weight=balanced` option of the scikit-learn classifiers, which is a form of cost-sensitive learning: the loss function is weighted inversely proportional to the class frequency, so errors on the minority class (Churn = Yes) are penalized more during training. This option does not require any modification of the dataset and is therefore simpler to apply than the resampling techniques.

Compared to the works above, this paper does not aim to reach a new state of the art on the dataset. The objective is more modest and complementary. Instead of testing many algorithms within similar families, as in Lalwani et al., or focusing on big-data implementations, as in Ahmad et al., this work compares one representative classifier per model family (linear, tree, ensemble, neural) under a single CRISP-DM pipeline. This design makes it possible to discuss the trade-offs between the families rather than between specific hyperparameter choices. The results obtained are in line with what has been reported on this dataset, which confirms that the pipeline is correctly implemented and that the chosen evaluation protocol is consistent with the published baselines.

3. Proposed Method

The work follows the CRISP-DM methodology [2], which divides the data mining process into six phases: business understanding, data understanding, data preparation, modeling, evaluation, and deployment. This section describes the first five (deployment is out of scope for this assignment).

3.1. Business and Data Understanding

The business goal is to support the retention team by identifying customers who are likely to churn in the near future, so that targeted offers can be sent to them before they actually leave. The corresponding machine learning problem is binary classification, with the customer profile as input and Churn = {Yes, No} as target.

The dataset is the public IBM Telco Customer Churn dataset [1], with 7,043 records and 21 columns. The target column is Churn. The other 20 columns include three numeric features (tenure in months, monthly charges and total charges), one binary numeric feature (SeniorCitizen, encoded as 0/1), one identifier (customerID), and fifteen

categorical features describing demographics, the contract and the optional services subscribed by the customer.

The initial data exploration reveals three relevant facts. First, the dataset is imbalanced: about 73% of the customers did not churn and about 27% did. Second, the TotalCharges column is stored as text and contains 11 empty values, all corresponding to customers with tenure = 0 (brand new customers who have not been billed yet). Third, the strongest single predictors of churn (visible in the bivariate analysis) are the contract type, the tenure, the internet service and the payment method. For example, the churn rate is about 43% for month-to-month contracts and only about 3% for two-year contracts.

3.2. Data Preparation

The data preparation step applies five sub-activities on the dataset.

Selection. The customerID column is dropped because it has one unique value per row and no predictive information. All other columns are kept.

Cleansing. The TotalCharges column is converted to a numeric type using `pd.to_numeric` with `errors='coerce'`, which turns the 11 empty strings into NaN values. These 11 rows (about 0.16% of the data) all correspond to tenure = 0 and Churn = No. Since their TotalCharges value is not really missing but undefined (the customer has not been billed yet), and since they represent a very small fraction of the data, these rows are dropped. A check based on the IQR rule with $k = 1.5$ finds no outliers in the three numeric features.

Construction (encoding). Since most classifiers require numeric input, the categorical features are encoded with three different strategies depending on their nature. The target Churn is mapped to 1/0 for Yes/No. The binary Yes/No columns (and the gender column, mapped as Male=1 / Female=0) are converted to 0/1. The three-value service columns (MultipleLines, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies), which can take the values “Yes”, “No” or “No internet/phone service”, are simplified to binary Yes/No: the third value is treated as “No”, since it is already captured by the PhoneService and InternetService columns. The Contract column has a natural order (Month-to-month < One year < Two year) and is therefore encoded as ordinal 0/1/2. Finally, the two nominal multi-value columns (InternetService and PaymentMethod) are encoded with one-hot encoding via pandas `get_dummies`.

Integration. Not applicable, since the data comes from a single table.

Formatting. The dataset is split into 2/3 training and 1/3 test with `train_test_split`. The test set is used only for the final evaluation, never for hyperparameter selection. For the models that are sensitive to feature scale (Logistic Regression and MLP), a `StandardScaler` is added in front of the classifier inside a scikit-learn Pipeline. This way the scaler is fit only on the training portion in each cross-validation fold, so no information leaks from the test set

to the training process. The tree-based models do not need scaling, since the tree splits are independent of feature scale.

3.3. Modeling

Five classifiers from different model families are compared:

- **Dummy** (majority class baseline). It always predicts the most frequent class (“No”). It is included as a sanity check: any useful model has to clearly beat this baseline.
- **Logistic Regression.** A simple linear probabilistic classifier, used here as the linear baseline. The regularization parameter C is tuned over $\{0.1, 1, 10\}$.
- **Decision Tree.** A standard CART classifier. The maximum depth and the minimum number of samples per leaf are tuned.
- **Random Forest.** An ensemble of decision trees aggregated by bagging. The number of estimators and the maximum depth are tuned.
- **Multi-Layer Perceptron (MLP).** A small feed-forward neural network from scikit-learn, with one or two hidden layers (50 or 100 units), ReLU activation and the Adam optimizer. Early stopping is enabled to limit overfitting.

For all the models except the Dummy, the hyperparameters are selected with `GridSearchCV` using 5-fold cross-validation on the training set. The optimization metric is the F1 score on the positive class (Churn = 1), to push the search toward models that perform well on the minority class.

The class imbalance is handled by setting `class_weight=balanced` in Logistic Regression, the Decision Tree and the Random Forest. This option weights the loss function inversely proportional to the class frequency, so the minority class (Churn = Yes) receives a higher weight during training. The MLP classifier does not support this option, so the F1 optimization criterion alone drives the search for that model.

3.4. Evaluation Protocol

Each model is evaluated on the held-out test set. The reported metrics are Accuracy, Precision (positive class), Recall (positive class), F1 (positive class) and ROC-AUC. As discussed in Section 2, accuracy alone is misleading on imbalanced data, so the main quality metrics for this work are F1 and ROC-AUC. The confusion matrices and the ROC curves are also computed, and the feature importance produced by the Random Forest is used to identify which customer attributes drive the predictions the most.

4. Results

Table 1 reports the five quality metrics computed on the held-out test set for each of the five classifiers. Precision, Recall and F1 are reported for the positive class (Churn = Yes), since this is the class of interest for the retention team.

TABLE 1. TEST SET PERFORMANCE.

Model	Acc.	Prec.	Recall	F1	ROC-AUC
Logistic Regression	0.745	0.508	0.801	0.622	0.842
Random Forest	0.773	0.553	0.700	0.618	0.837
Decision Tree	0.737	0.499	0.749	0.599	0.819
MLP	0.797	0.641	0.510	0.568	0.837
Dummy	0.738	0.000	0.000	0.000	0.500

4.1. Model comparison

Logistic Regression and Random Forest obtain the best F1 (about 0.62), with values very close to each other. Their ROC-AUC is around 0.84, which matches the result reported by Lalwani et al. [4] on the same dataset with boosting methods. Between the two, Logistic Regression is preferable in this case because it is a simpler model, faster to train and easier to interpret, while providing essentially the same performance. The Decision Tree comes third with an F1 of about 0.60 and a slightly lower ROC-AUC of about 0.82, which is a reasonable result considering that it is a single tree without any ensemble.

The Dummy baseline confirms a point that was already clear from the data understanding step: accuracy alone is misleading on this dataset. The Dummy achieves 0.738 accuracy (the proportion of the majority class) without predicting any churner, so its F1 is zero and its ROC-AUC is 0.5. The fact that all the trained models obtain F1 well above zero and ROC-AUC above 0.8 shows that they have actually learned a non-trivial pattern, even if their accuracy is only slightly above the Dummy.

An interesting observation is that the MLP has the highest accuracy (0.797) but the lowest Recall among the trained models (0.510). It achieves a higher accuracy by being more conservative and predicting “No” more often, but as a consequence it misses about half of the actual churners. From a business point of view this is the worst trade-off in the table. In a churn problem, the cost of a False Negative (a customer who actually leaves but the model does not flag) is much higher than the cost of a False Positive (the model flags a customer who actually stays, so the company wastes one retention offer). The lost lifetime revenue of a real churner is typically in the order of several hundred euros, while a wasted retention call costs only a few tens of euros. So missing churners is much more expensive than wasting offers, and a model with high precision but low recall is the wrong choice for this kind of problem. The models trained with `class_weight=balanced` (Logistic Regression, the Decision Tree and the Random Forest) show the opposite behavior: they sacrifice some precision and accuracy to obtain a higher recall, which is the right trade-off for a churn application.

4.2. Confusion matrices and ROC curves

The confusion matrices of the three best models (Logistic Regression, Random Forest and Decision Tree)

share a common pattern. The True Negative rate is high (most non-churners are correctly classified as non-churners), the True Positive rate is also high thanks to the `class_weight=balanced` option (around 70–80% of the real churners are detected), but the False Positive rate is non-negligible (around 25–30% of non-churners are incorrectly flagged as churners). This pattern is the direct consequence of the design choice of optimizing for recall on the positive class at the price of more false alarms. As discussed in the previous subsection, this is the desirable trade-off for a churn problem.

The ROC curves of the trained models are close to each other and clearly above the diagonal random-classifier line. Logistic Regression and Random Forest are nearly indistinguishable, while the Decision Tree curve is slightly below them, and the MLP curve sits in between despite the lower F1. The Dummy classifier sits exactly on the diagonal, as expected. The visible separation between the model curves and the diagonal in the low-FPR region is the practically useful part: it shows that the models can identify customers with a high churn probability while still keeping the false-positive rate at an acceptable level. This is what allows the retention team to rank customers by risk and contact the top portion of the list within a fixed budget.

4.3. Feature importance and business insights

The feature importance produced by the Random Forest highlights the same drivers that were already visible in the bivariate analysis of Section 3. The most important features are the contract type, the customer tenure, the monthly and total charges, and the `InternetService` and `PaymentMethod` one-hot indicators. In particular, the combinations of features that point most strongly to churn are month-to-month contract, short tenure, fiber optic internet and electronic check payment. These are the customers that the retention team should focus on first. On the other side, gender and `PhoneService` have almost no importance, which is consistent with the very flat churn rate observed for these features in the data (around 27% for both genders and for both values of `PhoneService`).

From a business point of view, these results have a direct interpretation. The contract type is the most actionable feature: encouraging customers to switch from month-to-month to a one-year or two-year contract should substantially reduce their churn probability. The tenure effect implies that the first months after subscription are the most critical, and that onboarding programs and welcome offers may have a strong protective effect. The `InternetService` and `PaymentMethod` findings also suggest a more focused operational action: investigating why fiber optic and electronic-check customers churn at higher rates may reveal service quality or billing issues that can be addressed independently of the classifier.

4.4. Stability check and comparison with literature

To check that the choice of train/test split does not significantly affect the conclusions, the whole pipeline was also run with test sizes of 20% and 25%, in addition to the 33% reported in Table 1. The differences in F1 and ROC-AUC are below 0.02 across the three configurations, and the model ranking is the same in all of them (Logistic Regression and Random Forest at the top, Decision Tree third and MLP last among the trained models). This indicates that the results are stable and not the consequence of a lucky split.

Overall, the numerical results are aligned with the recent literature on this dataset. The ROC-AUC of about 0.84 obtained here is similar to the value reported by Lalwani et al., which suggests that the dataset has a natural performance ceiling around 0.85 ROC-AUC. This ceiling is probably due to important churn drivers that are simply not present in the available features, such as customer-care interactions, competitor offers and perceived service quality.

5. Conclusions

This paper presented a churn prediction pipeline on the IBM Telco Customer Churn dataset, built following the CRISP-DM methodology. Five classifiers from different families were compared under a single pipeline. Logistic Regression and Random Forest obtained the best results (F1 around 0.62 and ROC-AUC around 0.84), aligned with what the recent literature reports on the same dataset. The most important features for the prediction are the contract type, the tenure, the internet service and the payment method. Customers with a month-to-month contract, short tenure, fiber optic internet and electronic check payment are the most at risk and should be the first targets for the retention campaigns.

The work has some limitations. The dataset is a single snapshot from one company, so the results may not generalize to other operators without retraining. The data does not include any time information beyond the tenure value, which prevents any sequential or time-series analysis. The performance ceiling of about 0.85 ROC-AUC also suggests that important churn drivers, such as competitor offers or quality of service issues, are simply not in the available features. Several directions are possible for future work. AdaBoost is one of the methods that Lalwani et al. tested on the same dataset, so it would be a natural extension to add it to the comparison and see if it gives better results than Random Forest. Other techniques to handle the class imbalance could also be tested and compared against `class_weight`, to check if the choice of imbalance strategy has a significant effect on the results.

References

[1] IBM Cognos Analytics, *Telco Customer Churn* [dataset], 2018. [Online]. Available: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

[2] C. Shearer, "The CRISP-DM model: the new blueprint for data mining," *Journal of Data Warehousing*, vol. 5, no. 4, pp. 13–22, 2000.

[3] A. K. Ahmad, A. Jafar, and K. Aljoumaa, "Customer churn prediction in telecom using machine learning in big data platform," *Journal of Big Data*, vol. 6, no. 1, art. 28, 2019.

[4] P. Lalwani, M. K. Mishra, J. S. Chadha, and P. Sethi, "Customer churn prediction system: a machine learning approach," *Computing*, vol. 104, no. 2, pp. 271–294, 2022.

[5] J. Burez and D. Van den Poel, "Handling class imbalance in customer churn prediction," *Expert Systems with Applications*, vol. 36, no. 3, pp. 4626–4636, 2009.